

排序算法

xbZhong

[本页PDF](#)

选择排序

- 分为待排序区和已排序区
- 顾名思义，选择一个最小值放到已排序区末尾

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  int *getranddata(int n)
4  {
5      int *arr = new int [n];
6      for(int i = 0;i < n;i++)
7      {
8          arr[i] = rand()%1000;
9      }
10     return arr;
11 }
12
13 bool check(int *arr,int n)
14 {
15     for(int i = 1;i < n;i++)
16     {
17         if(arr[i] < arr[i - 1])
18             return false;
19     }
20     return true;
21 }
22
23 void print(int *arr ,int n)
24 {
25     for(int i = 0;i < n;i++)
26     {
27         cout << arr[i] << " ";
28     }
29     cout << endl;
30     return;
31 }
32
33 void selection_sort(int *arr,int n)
34 {
35     for(int i = 0;i < n - 1;i++)
36     {
37         int ind = i;
38         for(int j = i + 1;j < n;j++)
39         {
40             if(arr[j] < arr[ind]) ind = j;
41         }
42         swap(arr[i],arr[ind]);
43     }
44     return;
45 }
46 int main()

```

```
47 {  
48     int n = 100;  
49     srand((unsigned)time(NULL));  
50     int *arr = getranddata(n);  
51     selection_sort(arr,n);  
52     print(arr,n);  
53     cout << check(arr,n) << endl;  
54     delete arr;  
55     return 0;  
56 }
```

插入排序

- 分为待排序区和已排序区
- 每次将待排序区第一个元素插入到已排序区中某个位置去
- 从已排序区最后一个元素开始不断和已排序区元素比较，直到下一个元素比插入元素小

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  int *getranddata(int n)
4  {
5      int *arr = new int [n];
6      for(int i = 0;i < n;i++)
7      {
8          arr[i] = rand()%1000;
9      }
10     return arr;
11 }
12
13 bool check(int *arr,int n)
14 {
15     for(int i = 1;i < n;i++)
16     {
17         if(arr[i] < arr[i - 1])
18             return false;
19     }
20     return true;
21 }
22
23 void print(int *arr ,int n)
24 {
25     for(int i = 0;i < n;i++)
26     {
27         cout << arr[i] << " ";
28     }
29     cout << endl;
30     return;
31 }
32
33 void insert_sort(int *arr,int n)
34 {
35     for(int i = 1;i < n;i++)
36     {
37         int j = i;
38         while(j > 0 && arr[j - 1] > arr[j])
39         {
40             swap(arr[j - 1],arr[j]);
41             j -= 1;
42         }
43     }
44     return;
45 }
46 int main()

```

```

47     {
48         int n = 100;
49         srand((unsigned)time(NULL));
50         int *arr = getranddata(n);
51         insert_sort(arr,n);
52         print(arr,n);
53         cout << check(arr,n) << endl;
54         delete arr;
55         return 0;
56     }

```

选择排序执行次数是插入排序的两倍

- 选择排序和插入排序(计算执行次数的期望)时间复杂度都为 $O(n^2)$
- 底层cpu对于数组遍历有加速效果，对于数据交换没有优化
- 选择排序遍历次数多，数据交换少，插入排序数据交换多，因此他们执行时间差不多

无监督的插入排序

- 和插入排序区别就是找到全局最小值将其放到第一位，后续while判断条件就不用 $j>0$
- 执行 $j>0$ 这个判断条件次数和交换数据的数量级一致，都是 $O(n^2)$
- 增加了 $O(n)$ 的查找最小值次数，减少了 $O(n^2)$ 的判断条件执行次数，优化了算法

```

1  void unsupervise_insert_sort(int *arr,int n) //无监督的插入排序
2  {
3      int ind = 0;
4      for(int i = 1;i < n;i++)
5      {
6          if(arr[ind] > arr[i]) ind = i;
7      }
8      while(ind > 0)
9      {
10         swap(arr[ind],arr[ind - 1]);
11         ind -= 1;
12     }
13     for(int i = 1;i < n;i++)
14     {
15         int j = i;
16         while( arr[j - 1] > arr[j])
17         {
18             swap(arr[j - 1],arr[j]);
19             j -= 1;
20         }
21     }
22     return;
23 }

```

希尔排序

- 设计一个步长序列
- 按照步长对序列进行分组，每组采用插入排序
- 直到执行到步长为1为止
- 希尔排序的效率和步长序列紧密相关 $> O(n^2)$ (最坏时间复杂度) 希尔增量序列: $n/2, n/4, n/8, n/16, \dots > O(n^{1.5})$ (最坏时间复杂度) Hibbard增量序列: $1, 3, 7, \dots, 2^{k-1}$

希尔增量

```
1 void shell_sort(int *arr,int n)
2 {
3     int k = 2,step;
4     do
5     {
6         step = n / k == 0 ? 1:n / k;
7         for(int i = 0;i < step;i++) //执行step次是因为每隔step，元素就是一组，要遍历完不
同组
8             unsupervise_insert_sort(arr,n,step);
9         k *= 2;
10    }while(step != 1);
11    return;
12 }
```

Hibbard增量

```
1 void shell_sort_hibbard(int *arr,int n)
2 {
3     int step = 1;
4     while(step <= n / 2) step = step * 2 + 1;
5     do
6     {
7         step /= 2;
8         for(int i = 0;i < step;i++)
9             unsupervise_insert_sort(arr,n,step);
10    }while(step > 1);
11    return ;
12 }
```

冒泡排序

- 创建个变量cnt去判断第一次扫描数组时是否有序，有序则直接break

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  void bubble_sort(int *arr,int n)
4  {
5      int cnt;
6      for(int i = n;i > 0;i--)
7      {
8          cnt = 0;
9          for(int j = 1;j < i;j++)
10         {
11             if(arr[j] >= arr[j - 1]) continue;
12             swap(arr[j],arr[j - 1]);
13             cnt += 1;
14         }
15         if(cnt == 0) break;
16     }
17     return ;
18 }
19
20 int *getranddata(int n)
21 {
22     int *arr = new int [n];
23     for(int i = 0;i < n;i++)
24         arr[i] = rand() % 1000;
25     return arr;
26 }
27
28 int check(int *arr ,int n)
29 {
30     for(int i = 1;i < n;i++)
31         if(arr[i] < arr[i - 1]) return 0;
32     return 1;
33 }
34
35 void print(int *arr,int n)
36 {
37     for(int i = 0;i < n;i++)
38         cout << arr[i] << " ";
39     cout << endl;
40     return ;
41 }
42 int main()
43 {
44     srand((unsigned)time(NULL));
45     int n = 100;
46     int *arr = getranddata(n);

```

```
47         bubble_sort(arr,n);
48         print(arr,n);
49         cout << check(arr,n) << endl;
50         delete arr;
51         return 0;
52     }
```

快速排序

- 找到一个基准值，利用其将数组分成两个区域
- 一区域的值比基准值小，另一区域的值比基准值大
 - 以数组第一个元素为基准值
 - 使用两个指针，一个指向头部，一个指向尾部
 - 尾指针移动找到比基准值小的值，将其移动到基准值所在位置
 - 头指针移动找到比基准值大的值，将其移动到尾指针所在位置
 - 头指针尾指针交替移动直到它们重叠，重叠位置就是基准值位置
- 后对两区域再执行上述操作(递归)


```

1      #include<bits/stdc++.h>
2      using namespace std;
3
4      void quick_sort(int *arr ,int l,int r) //左闭右开
5      {
6          if(r - l <= 2) //边界条件
7          {
8              if(r - l <= 1) return;
9              if(arr[l] > arr[l + 1]) swap(arr[l],arr[l + 1]);
10             return ;
11         }
12         //快排核心
13         int x = l,y = r - 1,z = arr[l];
14         while(x < y)
15         {
16             while(x < y && z <= arr[y]) y--;
17             if(x < y) arr[x] = arr[y];
18             while(x < y && z >= arr[x]) x++;
19             if(x < y) arr[y] = arr[x];
20         }
21         arr[x] = z;
22         quick_sort(arr,l,x);
23         quick_sort(arr,x+1,r);
24     }
25
26
27     int *getranddata(int n)
28     {
29         int *arr = new int [n];
30         for(int i = 0;i < n;i++)
31             arr[i] = rand() % 1000;
32         return arr;
33     }
34
35     int check(int *arr ,int n)
36     {
37         for(int i = 1;i < n;i++)
38             if(arr[i] < arr[i - 1]) return 0;
39         return 1;
40     }
41
42
43     void print(int *arr,int n)
44     {
45         for(int i = 0;i < n;i++)
46             cout << arr[i] << " ";

```

```

47         cout << endl;
48         return ;
49     }
50
51     int main()
52     {
53         int n = 100;
54         int *arr = getranddata(n);
55         quick_sort(arr,0,n);
56         print(arr,n);
57         cout << check(arr,n) << endl;
58         delete arr;
59         return 0;
60     }

```

快速排序优化

- 用头尾指针找到一对元素，直接交换元素位置(v1)
- 从基准值入手，找到适合的基准值(三点取中法)(v2)
- 减少递归次数，对右半部分使用循环进行操作(v3)
- 对于少量数据的区域采用无监督插入排序(v4) **原则：后半区间的元素大小大于等于前半区间的元素大小** > 对于while(x <= y) 需要加上等于号,否则当x=y时，它们指向的可能不是基准值，此时左区间为(l ~ y+1),右区间为(x ~ r)才可得出正确排序结果。 **但是这样效率降低，因为要对x和y同时指向的元素进行两次排序** > 对于swap(arr[x++], arr[y--]) x++与y--是防止x和y同时指向基准值会导致死循环 > 对于if(x <= y) 加上等于号是防止x=y时，x与y同时指向基准值造成死循环 > 对于while(z < arr[y]) 不加等于号是为了防止内存溢出，无限次调用系统栈，如数据1， 1， 1， 2， 2， 2， 5， 4， 1， 1， 以1为基准值

```

1
2      #include<bits/stdc++.h>
3      using namespace std;
4      void unsupervise_insert_sort(int *arr,int l,int r)
5      {
6          int ind = l;
7          for(int i = l + 1;i < r;i++)
8          {
9              if(arr[ind] > arr[i]) ind = i;
10         }
11         while(ind > l)
12         {
13             swap(arr[ind],arr[ind - 1]);
14             ind -= 1;
15         }
16         for(int i = l + 1;i < r;i++)
17         {
18             int j = i;
19             while(arr[j - 1] > arr[j])
20             {
21                 swap(arr[j - 1],arr[j]);
22                 j -= 1;
23             }
24         }
25         return;
26     }
27
28     void print(int *arr,int n)
29     {
30         for(int i = 0;i < n;i++)
31             cout << arr[i] << " ";
32         cout << endl;
33         return ;
34     }

```

- 标准快排

```

1 void quick_sort(int *arr ,int l,int r) //左闭右开
2 {
3     if(r - l <= 2) //边界条件
4     {
5         if(r - l <= 1) return;
6         if(arr[l] > arr[l + 1]) swap(arr[l],arr[l + 1]);
7         return ;
8     }
9     //快排核心
10    int x = l,y = r - 1,z = arr[l];
11    while(x < y)
12    {
13        while(x < y && z <= arr[y]) y--;
14        if(x < y) arr[x] = arr[y];
15        while(x < y && z >= arr[x]) x++;
16        if(x < y) arr[y] = arr[x];
17    }
18    arr[y] = z;
19    quick_sort(arr,l,x);
20    quick_sort(arr,x+1,r);
21 }

```

- 直接交换左右指针指向的值

```

1 void quick_sort_v1(int *arr ,int l,int r) //直接交换左右指针指向的值
2 {
3     if(r - l <= 2) //边界条件
4     {
5         if(r - l <= 1) return;
6         if(arr[l] > arr[l + 1]) swap(arr[l],arr[l + 1]);
7         return ;
8     }
9     //快排核心
10    int x = l,y = r - 1,z = arr[l];
11    while(x <= y)
12    {
13        while(z < arr[y]) y--;
14        while(z > arr[x]) x++;
15        if(x <= y)
16            swap(arr[x++], arr[y--]);
17    }
18    quick_sort_v1(arr,l,x);
19    quick_sort_v1(arr,x,r);
20 }

```

- 优化基准值

```
1      int way(int a,int b,int c)
2      {
3          if(a > b) swap(a,b);
4          if(a > c) swap(a,c);
5          if(b > c) swap(b,c);
6          return b;
7      }
8
9      void quick_sort_v2(int *arr ,int l,int r) //优化基准值
10     {
11         if(r - l <= 2) //边界条件
12         {
13             if(r - l <= 1) return;
14             if(arr[l] > arr[l + 1]) swap(arr[l],arr[l + 1]);
15             return ;
16         }
17         //快排核心
18         int x = l,y = r - 1;
19         int z = way(arr[l],arr[r - 1],arr[(l + r) / 2]);
20         while(x < y)
21         {
22             while( z < arr[y]) y--;
23             while( z > arr[x]) x++;
24             if(x <= y)
25                 swap(arr[x++], arr[y--]); //x++与y--防止x, y指针遇到基准值
26             //造成死循环
27
28         }
29         quick_sort_v2(arr,l,y+1);
30         quick_sort_v2(arr,x,r);
31     }
```

- 减少递归次数，采用循环方式

式

```
1
2 void quick_sort_v3(int *arr ,int l,int r) //减少递归次数，对右半区间采用循环方
3 {
4     while(l < r)
5     {
6         if(r - l <= 2) //边界条件
7         {
8             if(r - l <= 1) return;
9             if(arr[l] > arr[l + 1]) swap(arr[l],arr[l + 1]);
10            return ;
11        }
12        //快排核心
13        int x = l,y = r - 1;
14        int z = way(arr[l],arr[r - 1],arr[(l + r) / 2]);
15        while(x <= y)
16        {
17            while(z < arr[y]) y--;
18            while(z > arr[x]) x++;
19            if(x <= y)//可能会出现头尾指针过度移动的情况
20                swap(arr[x++], arr[y--]);
21        }
22        quick_sort_v3(arr,l,x);
23        l = x;
24    }
25    return ;
26 }
```

- 对于少量数据采用无监督插入排序

```

1      void __quick_sort_v4(int *arr ,int l,int r) //左闭右开
2      {
3          while(r - l > 16)
4          {
5              //快排核心
6              int x = l,y = r - 1,z = arr[l];
7              while(x <= y)
8              {
9                  while(z < arr[y]) y--;
10                 while(z > arr[x]) x++;
11                 if(x <= y) //可能会出现头尾指针过度移动的情况
12                     swap(arr[x++] , arr[y--]);
13             }
14             __quick_sort_v4(arr,l,x);
15             l = x;
16         }
17         return ;
18     }
19
20
21     void quick_sort_v4(int *arr ,int l,int r) //左闭右开
22     {
23         __quick_sort_v4(arr,l,r);
24         unsupervise_insert_sort(arr,l,r);
25     }

```

- 上述代码实现时x与y会错开，即x会在y的后面，y会在x的前面

归并排序

- 分治：将数组拆分成小数组进行排序
- 归并：将排完序的小数组归并成为一个大的有序数组
- 采用的是递归的思想
- 分成两个小数组，将两个小数组的值按从小到大放进temp数组里面，再进行拷贝
- 时间复杂度稳定，为 $O(n \log n)$

```

1
2     void merge_sort(int *arr,int l,int r)
3     {
4         if(r - l <= 1) return ;
5         int mid = (l + r) / 2;
6         merge_sort(arr,l,mid);
7         merge_sort(arr,mid,r);
8         int p1 = l , p2 = mid , k = 0;
9         while(p1 < mid || p2 < r)
10        {
11            if(p2 == r || (p1 < mid && arr[p1] <= arr[p2]))
12                temp[k++] = arr[p1++];
13            else
14                temp[k++] = arr[p2++];
15        }
16        for(int i = l;i < r;i++)
17            arr[i] = temp[i - l];
18        return ;
19    }

```

基数排序

相同数字相对位置不变 * 选择数字的某个位置进行排序(个位，十位，等等) * 求得前缀和数列 * 从后向前扫描数组 * 时间复杂度为 $O(n)$ > 对于一个整型数据，其最大数值为 2^{32} 。因此我们选取 2^{16} 为基数。这样无论对于任何一个整型数据，我们最多只需要排两轮。轮数为： $\lceil \log_{65536} m \rceil$ * 进行完一轮基数排序后，要将temp数据拷贝到arr再进行下一轮排序才能得到正确结果 *


```

1      #include<bits/stdc++.h>
2      using namespace std;
3
4      void print(int *arr,int l,int r)
5      {
6          for(int i = l;i < r;i++)
7          {
8              if(i) cout << " ";
9              cout << arr[i];
10         }
11         cout << endl;
12         return;
13     }
14
15
16     int *getnewdata(int l ,int r)
17     {
18         srand((unsigned)time(NULL));
19         int *arr = new int[r - l];
20         for(int i = l;i < r;i++)
21         {
22             arr[i] = rand() % 100;
23         }
24         return arr;
25     }
26     #define K 10
27     void radix_sort(int *arr,int l,int r)
28     {
29         int *cnt = new int[K];
30         int *temp = new int[r - l];
31         memset(cnt,0,sizeof(int) * K);
32         for(int i = l;i < r;i++) cnt[arr[i] % K] += 1;
33         for(int i = 1;i < K;i++) cnt[i] += cnt[i - 1]; //求前缀和数组
34         for(int i = r - 1;i >= l;i--) temp[--cnt[arr[i] % K]] = arr[i];
35         memcpy(arr + l,temp,sizeof(int) * (r - l));
36         memset(cnt,0,sizeof(int) * K);
37         for(int i = l;i < r;i++) cnt[arr[i] / K] += 1;
38         for(int i = 1;i < K;i++) cnt[i] += cnt[i - 1]; //求前缀和数组
39         for(int i = r - 1;i >= l;i--) temp[--cnt[arr[i] / K]] = arr[i];
40         memcpy(arr + l,temp,sizeof(int) * (r - l));
41         delete cnt,temp;
42         return ;
43     }
44
45
46     bool check(int *arr,int l,int r)

```

```

47     {
48         for(int i = l + 1; i < r; i++)
49         {
50             if(arr[i] < arr[i - 1]) return false;
51         }
52         return true;
53     }
54     #define n 10000
55     int main()
56     {
57         int *arr = getnewdata(0,n);
58         radix_sort(arr,0,n);
59         //print(arr,0,n);
60         cout << check(arr,0,n);
61         delete arr;
62         return 0;
63     }

```

sort函数用法

- 函数参数：第1个参数是排序区间的头部，第2个参数是排序区间的尾部(是左闭右开的)
- 默认升序
- 若要从大到小排序，则在加上第3个参数，模板函数greater<待排序元素的类型>()
- 对vector调用.end()时，其返回的是最后一个元素的下一个位置 ##### 自定义排序方式

1. 实现降序排序

```

1     bool cmp(int a,int b)
2     {
3         return a > b; //前一个参数大于后一个参数返回true，也就是降序
4     }
5     sort(arr,arr + 10;cmp);

```

2. 实现多维度排序

```

1     bool cmp(struct a,struct b)
2     {
3         if(a.x != b.x) return a.x > b.x;
4         return a.y < b.y;
5     }

```

- 对结构体中的x进行降序排序，对y进行升序排序

两数之和

1. 两数之和

已解答 

简单



相关标签



相关企业



提示

Ac

给定一个整数数组 `nums` 和一个整数目标值 `target`，请你在该数组中找出 **和为目标值** `target` 的那 **两个** 整数，并返回它们的数组下标。

你可以假设每种输入只会对应一个答案。但是，数组中同一个元素在答案里不能重复出现。

你可以按任意顺序返回答案。

示例 1:

输入: `nums = [2,7,11,15]`, `target = 9`

输出: `[0,1]`

解释: 因为 `nums[0] + nums[1] == 9`，返回 `[0, 1]`。

示例 2:

输入: `nums = [3,2,4]`, `target = 6`

输出: `[1,2]`

* 题目要求的是返回原数组的元素下标，意味着我们不能对数组进行排序 * 因此我们创建一个下标数组，根据原数组元素大小对下标数组进行排序 * 随后使用双指针，一个指向原数组最小值，一个指向最大值(最小值下标就是排序后的下标数组的第一位，最大值同理) * 比较两值和与`target`大小，移动`p1`或`p2` * 在最后压入`arr`即可

```
1      class Solution {
2      public:
3          vector<int> twoSum(vector<int>& nums, int target) {
4              vector<int> arr,ans;
5              for(int i = 0;i < nums.size();i++)
6                  ans.push_back(i);
7              sort(ans.begin(),ans.end(),[](int i,int j)->bool
8                  {
9                      return nums[i] < nums[j];
10                 });
11              int p1 = 0,p2 = nums.size() - 1;
12              while(nums[ans[p1]] + nums[ans[p2]] != target)
13              {
14                  if(nums[ans[p1]] + nums[ans[p2]] > target)
15                      p2--;
16                  if(nums[ans[p1]] + nums[ans[p2]] < target)
17                      p1++;
18              }
19              arr.push_back(ans[p1]);
20              arr.push_back(ans[p2]);
21              return arr;
22          }
23      };
```

排序链表

148. 排序链表

已解答 ✓

中等

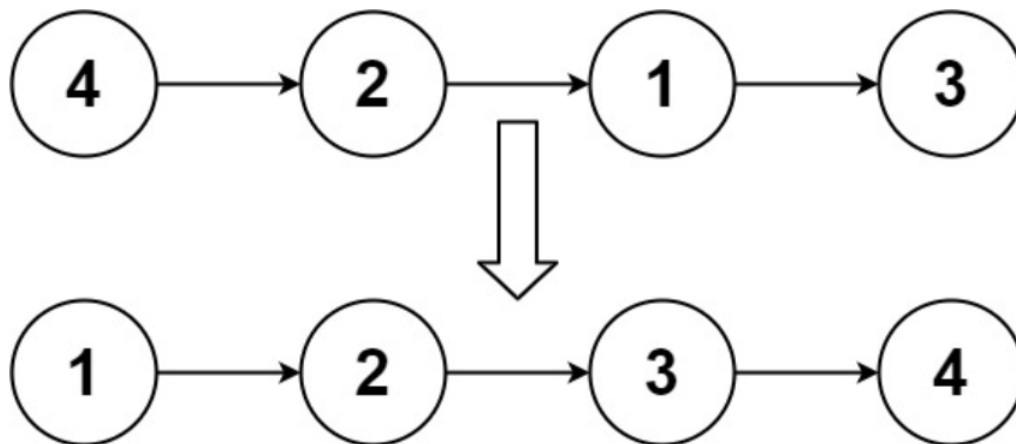
🏷 相关标签

🔒 相关企业

⌵ 难度

给你链表的头结点 `head`，请将其按 **升序** 排列并返回 **排序后的链表**。

示例 1:



输入: `head = [4,2,1,3]`

输出: `[1,2,3,4]`

* 快速排序(类似快排，其不是在原链表进行分区，而是将值拷贝到新链表，再进行连接) * z 的取值要注意是右移(向下取整)，而不是除以(向0取整)

```

1      class Solution {
2      public:
3          ListNode* sortList(ListNode* head) {
4              if(head == NULL || head->next == NULL) return head;
5              int l = head->val, r = head->val, z;
6              ListNode *p = head, *q, *h1 = NULL, *h2 = NULL;
7              while(p) l = min(p->val, l), r = max(p->val, r), p = p->next;
8              z = (l + r) >> 1;
9              if(l == r) return head;
10             p = head;
11             while(p)
12             {
13                 q = p->next;
14                 if(p->val <= z)
15                 {
16                     p->next = h1;
17                     h1 = p;
18                 }
19                 else
20                 {
21                     p->next = h2;
22                     h2 = p;
23                 }
24                 p = q;
25             }
26             h1 = sortList(h1);
27             h2 = sortList(h2);
28             p = h1;
29             while(p->next) p = p->next;
30             p->next = h2;
31             return h1;
32         }
33     };

```

- 归并排序

```

1      class Solution {
2      public:
3          int getlength(ListNode *head)
4          {
5              int n = 0;
6              while(head) n+=1 , head = head->next;
7              return n;
8          }
9          ListNode *merge_sort(ListNode *head , int n)
10         {
11             if(n <= 1) return head;
12             int l = n / 2, r = n - l;
13             ListNode *p = head,*p1 = head,*p2,new_head;
14             for(int i = 1;i < l;i++) p = p->next;
15             p2 = p->next;
16             p->next = NULL;
17             p1 = merge_sort(p1,l);
18             p2 = merge_sort(p2,r);
19             p = &new_head,new_head.next = NULL;
20             while(p1 || p2)
21             {
22                 if(p2 == NULL || (p1 && p1->val < p2->val))
23                 {
24                     p->next = p1;
25                     p = p1;
26                     p1 = p1->next;
27                 }
28                 else
29                 {
30                     p->next = p2;
31                     p = p2;
32                     p2 = p2->next;
33                 }
34             }
35             return new_head.next;
36         }
37         ListNode* sortList(ListNode* head) {
38             int n = getlength(head);
39             return merge_sort(head, n);
40         }
41     };

```

合并两个有序数组

- 使用sort

```

1      class Solution {
2      public:
3          int t = 65536;
4          void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
5              if(n == 0) return;
6              for(int i = 0;i < n;i++) nums1[m + i] = nums2[i];
7                  sort(nums1.begin(),nums1.end());
8              return;
9          }
10     };

```

- 双指针

```

1
2      class Solution {
3      public:
4          void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
5              vector<int> arr;
6              arr.resize(nums1.size());
7              int p1 = 0 , p2 = 0,k = 0;
8              while(p1 != m || p2 != n )
9              {
10                 if(p2 == n || (p1 != m && nums1[p1] < nums2[p2]))
11                 {
12                     arr[k] = nums1[p1];
13                     p1++,k++;
14                 }
15                 else
16                 {
17                     arr[k] = nums2[p2];
18                     p2++,k++;
19                 }
20             }
21             for(int i = 0;i < (m + n);i++)
22             {
23                 nums1[i] = arr[i];
24             }
25             return ;
26         }
27     };

```

- 反向双指针
- 若想直接在nums1进行排序，则需要在nums1后部进行插入，在前面插入会使得数据被覆盖


```

1
2     class Solution {
3     public:
4         void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
5             int p1 = m - 1, p2 = n - 1, k = m + n - 1;
6             while(p1 >= 0 || p2 >= 0 )
7             {
8                 if(p2 == -1 || (p1 != -1 && nums1[p1] > nums2[p2]))
9                 {
10                     nums1[k] = nums1[p1];
11                     p1--, k--;
12                 }
13                 else
14                 {
15                     nums1[k] = nums2[p2];
16                     p2--, k--;
17                 }
18             }
19             return ;
20         }
21     };

```

合并两个有序链表

- 运用到了归并排序的思想

```

class Solution { public: ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) { ListNode *p, new_head; p =
&new_head; while(list1 || list2) { if(list2 == NULL || (list1 != NULL && list1->val < list2->val)) { p->next = list1; list1 = list1-
>next; p = p->next; } else { p->next = list2; list2 = list2->next; p = p->next; } } return new_head.next; } };

```

寻找两个正序数组的中位数

4. 寻找两个正序数组的中位数

已解答 

困难

 相关标签

 相关企业

Ax

给定两个大小分别为 m 和 n 的正序（从小到大）数组 `nums1` 和 `nums2`。请你找出并返回这两个正序数组的 **中位数**。

算法的时间复杂度应该为 $O(\log(m+n))$ 。

示例 1:

输入: `nums1 = [1,3]`, `nums2 = [2]`

输出: `2.00000`

解释: 合并数组 = `[1,2,3]` , 中位数 `2`

示例 2:

输入: `nums1 = [1,2]`, `nums2 = [3,4]`

输出: `2.50000`

解释: 合并数组 = `[1,2,3,4]` , 中位数 $(2 + 3) / 2 = 2.5$

alt text

```

1  class Solution {
2  public:
3      double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
4          int n = nums1.size(), m = nums2.size();
5          vector<int> temp(n + m);
6          int p1 = 0, p2 = 0, k = 0;
7          while(p1 != n || p2 != m )
8          {
9              if(p2 == m || (p1 != n && nums1[p1] < nums2[p2]))
10             {
11                 temp[k] = nums1[p1];
12                 p1++, k++;
13             }
14             else
15             {
16                 temp[k] = nums2[p2];
17                 p2++, k++;
18             }
19         }
20         int t = temp.size() / 2;
21         if(temp.size() % 2)
22             return temp[t];
23         return (temp[t] + temp[t - 1]) / 2.0;
24     }
25 };

```

存在重复元素

- 个人

```

1      class Solution {
2      public:
3          bool containsNearbyDuplicate(vector<int>& nums, int k) {
4              int p1 = 0, p2 = 1, n = nums.size();
5              while (p1 < n) {
6                  while (p2 < n && (p2 - p1) <= k) {
7                      if (nums[p1] == nums[p2])
8                          return true;
9                      p2++;
10                 }
11                 p1++;
12                 p2 = p1 + 1;
13             }
14             return false;
15         }
16     };
17
18 * 船长
19
20     class Solution {
21     public:
22         bool containsNearbyDuplicate(vector<int>& nums, int k) {
23             int n = nums.size();
24             vector<int> ind(n);
25             for(int i = 0; i < n; i++) ind[i] = i;
26             sort(ind.begin(), ind.end(), [&](int i, int j) -> bool
27             {
28                 return nums[i] < nums[j];
29             });
30             for(int i = 0; i < n - 1; i++)
31             {
32                 if(nums[ind[i]] != nums[ind[i + 1]]) continue;
33                 if(abs(ind[i] - ind[i + 1]) <= k) return true;
34             }
35             return false;
36         }
37     };

```

逆序对个数

#248. 逆序对个数

[描述](#)[提交](#)[自定义测试](#)[题解视频](#)[上一题](#)[下一题](#)[统计](#)

题目描述

输入 N 组数据，对每组数据输出逆序对个数。

输入

对于每组测试用例，第一行输入此组数据元素个数 x ，接下来 x 行，每行一个数，表示元素。（ $1 \leq x \leq 500000$ ）

当读入的元素个数 x 为零时，程序结束。

输出

对于每组测试用例，输出一个数，表示逆序对个数。

样例输入

```
5
9
1
0
5
4
3
1
2
3
0
```

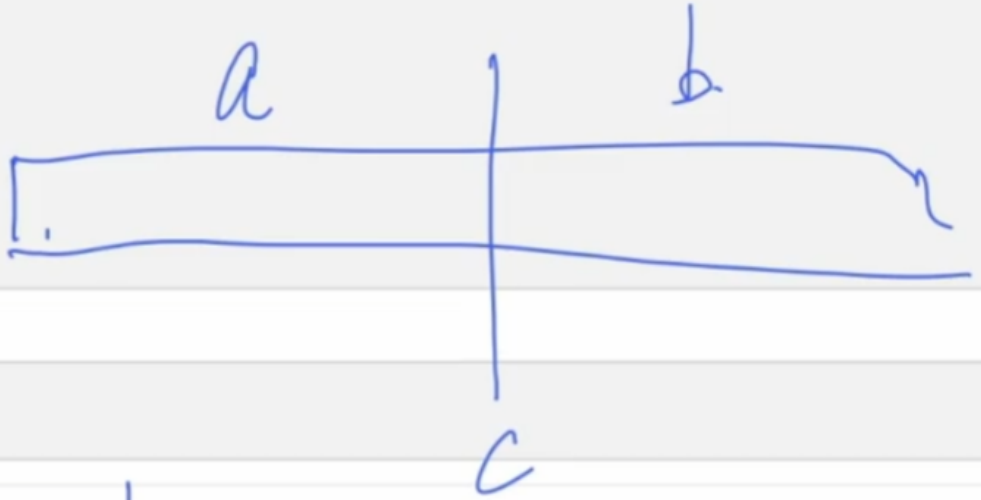
样例输出

```
6
0
```

* 可以分成a(左半部分的逆序对), b(右半部分的逆序对),c(横跨左右两区间的逆序对) * 对于a和b，可以使用递归来进行求解 * 对于a，b，其又可以分成三部分来求解逆序对，因此可使用递归来求解

lem/248

技术博客 信息学 笔试面试相关 MAC相关 生活相关 工商税务 动漫娱乐 人工智能 网站开发 科技信息 iCloud 新浪微博 腾讯微博 W 维基百科 百度 中国雅虎



$a + b + c$

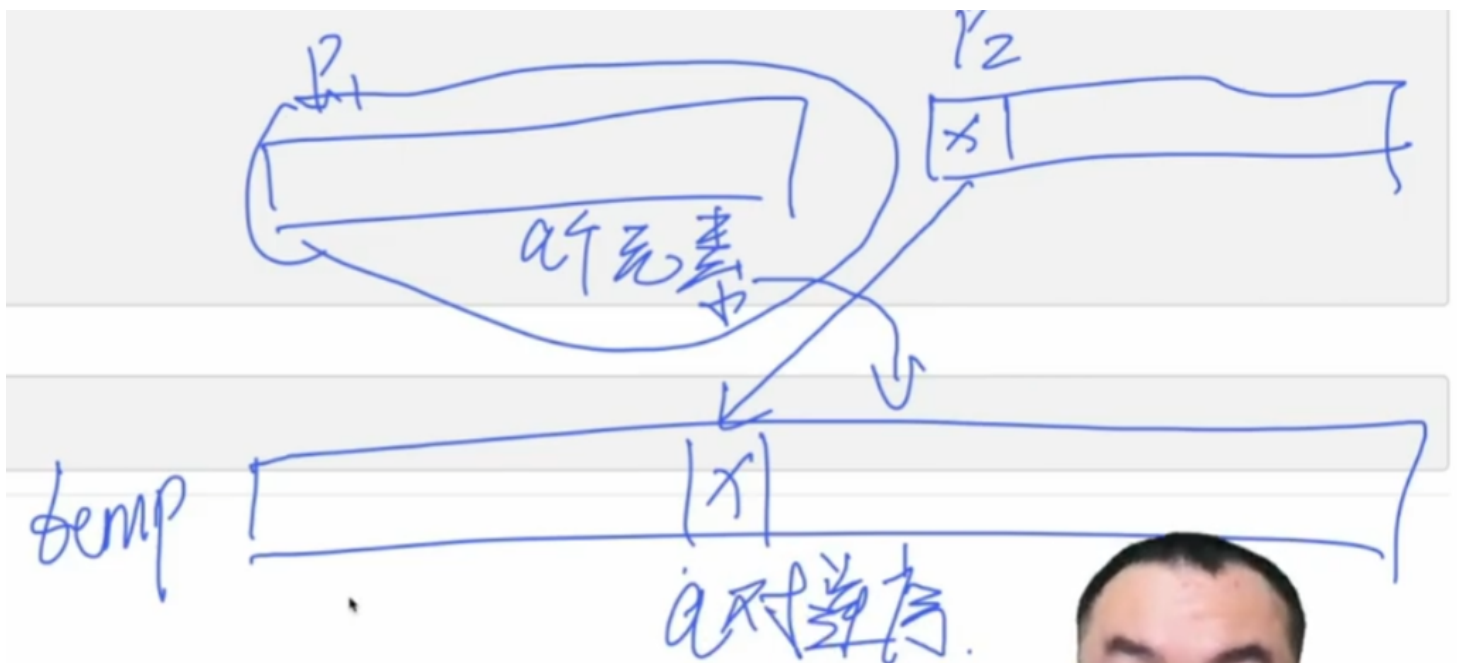
$1 \leq x \leq 500000$

递回

算法提升



使用到了归并排序的思想 * 在p1和p2中选取较小的元素放入temp，假设p2指向的元素被放入，则p1后面的a个元素与p2指向的元素为逆序对 * 要达到上述情况，则要保证左右两个区域的元素都是有序的，因此循环结束后要将temp里的元素赋给arr，以保证区域内元素有序



```

1      #include<bits/stdc++.h>
2      using namespace std;
3      int arr[500005],temp[500005];
4
5      long long merge_sort(int *arr,int l,int r)
6      {
7          if(r - l <= 1) return 0;
8          int mid = (r + l) / 2;
9          long long a = merge_sort(arr, l, mid);
10         long long b = merge_sort(arr, mid, r);
11         long long c = 0;
12         int p1 = l, p2 = mid, k = 0;
13         while(p1 != mid || p2 != r)
14         {
15             if(p2 == r || (p1 != mid && arr[p1] <= arr[p2]))
16                 temp[k++] = arr[p1++];
17             else
18             {
19                 temp[k++] = arr[p2++];
20                 c += (mid - p1);
21             }
22         }
23         for(int i = l;i < r;i++) arr[i] = temp[i - l]; //是为了让统计过的区域变得
有序
24         return a + b + c;
25     }
26     void solve(int n)
27     {
28         for(int i = 0;i < n;i++) cin >> arr[i];
29         cout << merge_sort(arr,0,n) << endl;
30         return;
31     }
32
33     int main()
34     {
35         int n;
36         while(1)
37         {
38             cin >> n;
39             if(n == 0) break;
40             solve(n);
41         }
42         return 0;
43     }

```

#251. 士兵

■ 描述

🔗 提交

➤ 自定义测试

📺 题解视频

📊 上一题

📊 下一题

📊 统计

题目描述

一些士兵站在矩阵的一些方格内，现要把他们移动到一横排，并连续地排成一队，士兵一次可以选择四个方向中的一个方向移动一格，求最少需要移动多少步才能完成要求。
即所有士兵的y坐标相同并且x坐标相邻。

输入

第一行输入一个正整数 n ，表示士兵的数量。（ $1 \leq n \leq 10000$ ）
接下来 n 行，每行两个数，代表第 i 个士兵所处位置的横纵坐标 X_i, Y_i 。（ $-10000 \leq X_i, Y_i \leq 10000$ ）

输出

输出最少移动步数。

* 最少移动步数为y方向上的移动步数加上x方向上的移动步数 * y方向上的移动步数为

$$\sum_{i=1}^n |y_i - Y|$$

*

$$\text{cost}_x = |x_i - (X + i)|$$

x_0 x_1 x_2 x_3

$\leq 10000, -10000 \leq X_i, Y_i \leq 10000$

x $x+1$ $x+2$ $x+3$

* x方向上的移动步数为

$$\sum_{i=1}^n |(x_i - i) - X|$$

* 要使得x方向上移动步数最少，则士兵相对位置是不变的 * 相当于做两次货仓选址

```
1      #include<bits/stdc++.h>
2      using namespace std;
3      int main()
4      {
5          int n;
6          cin >> n;
7          vector<int> x(n),y(n);
8          for(int i = 0;i < n; i++) cin >> x[i] >> y[i];
9          int X,Y,costx = 0,costy = 0;
10         sort(x.begin(),x.end());
11         for(int i = 0; i < n; i++) x[i] = x[i] - i;
12         sort(x.begin(),x.end());
13         sort(y.begin(),y.end());
14         X = x[n / 2];
15         Y = y[n / 2];
16         for(int i = 0;i < n; i++) costy += abs(y[i] - Y);
17         for(int i = 0;i < n; i++) costx += abs(x[i] - X);
18         cout << costx + costy << endl;
19         return 0;
20     }
```

国王游戏

时间限制:1 s 空间限制:256 MB

#256. 国王游戏

■ 描述 [🔗 提交](#) [🔗 自定义测试](#) [🔗 隔解视频](#)

[📊 上一题](#) [📊 下一题](#) [📊 统计](#)

题目描述

恰逢 H 国国庆,国王邀请 n 位大臣来玩一个有奖游戏。首先,他让每个大臣在左、右手上面分别写下一个整数,国王自己也在左、右手上各写一个整数。然后,让这 n 位大臣排成一排,国王站在队伍的最前面。排好后,所有的大臣都会获得国王奖赏的若干金币,每位大臣获得的金币数分别是:排在该大臣前面的所有人的左手上的数的乘积除以他自己右手上的数,然后向下取整得到的结果。

国王不希望某一个大臣获得特别多的奖赏,所以他想请你帮他重新安排一下队伍的顺序,使得获得奖赏最多的大臣,所获奖赏尽可能的少。注意,国王的位置始终在队伍的最前面。

输入

第一行包含一个整数 n , 表示大臣的人数。

第二行包含两个整数 a 和 b , 之间用一个空格隔开,分别表示国王左手和右手上的整数。(均小于 10000)

接下来 n 行, 每行包含两个整数 a 和 b , 之间用一个空格隔开, 分别表示每个大臣左手和右手上的整数。(均小于 10000)

输出

输出一个整数,表示重新排列后的队伍中获奖赏最多的大臣所获得的金币数。

* 对于序列1, 我们对其进行微扰 * 即交换 C_i 与 C_{i+1} 的位置得到序列2,其他位置的值不会变 * 为了满足题目所给的要求, 我们希望得到的序列2的最大值小于序列1的最大值 > 假设 C_{i+1} 是序列1的最大值, 则交换后可知 C'_{i+1} 小于 C_{i+1} , 为了让序列2的

最大值比序列1的最大值小，我们要让 C_{i+1} 大于 C'_i ，可以解得 $A_i * B_i \geq A_{i+1} * V_{i+1}$ ，上述得到的式子是需要进行交换才能得到更优解的，因此我们可以得到 $A_i * B_i \leq A_{i+1} * V_{i+1}$ 的排序顺序 > 而若 C_i 是序列1最大值，交换后序列2还有比其更大的

HZOJ-256：国王游戏

对序列，施加『微扰』，调换 i 与 $i+1$ 位置的大臣，观察序列前后的变化：

$$C_1 \quad C_2 \quad \cdots \quad C_i \quad C_{i+1} \quad \cdots \quad C_n$$

$$C_1 \quad C_2 \quad \cdots \quad C_{i+1}' \quad C_i' \quad \cdots \quad C_n$$

$$C_i = \prod_{j=0}^{i-1} A_j / B_i$$

$$C_i' = \prod_{j=0}^{i-1} A_j \times A_{i+1} / B_i$$

$$C_{i+1} = \prod_{j=0}^i A_j \times A_i / B_{i+1}$$

$$> C_{i+1}' = \prod_{j=0}^i A_j / B_{i+1}$$

《船说：算法》